

**LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 10**



Sultan Zulvi Risda

(105224034)

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA**

2026

1. Pendahuluan

Praktikum kali ini bertujuan untuk memahami arsitektur dan karakteristik dari tiga pilar utama Java Collections Framework (JCF), yaitu List, Set, dan Map, sebagai solusi penyimpanan data dinamis untuk mengatasi keterbatasan ukuran array tradisional yang kaku. Melalui studi kasus Sistem Manajemen Gudang berbasis Pemrograman Berorientasi Objek (PBO) murni, praktikum ini membatasi implementasi pada tiga kelas utama—Barang, SistemGudang, dan Main—dengan memanfaatkan HashMap sebagai basis data lokal untuk mempercepat pencarian barang berdasarkan ID, HashSet untuk menjamin keunikan nama kategori tanpa adanya duplikasi, serta ArrayList untuk mencatat kronologis riwayat transaksi secara terurut. Seluruh pengujian metode dilakukan secara terprogram langsung (hardcode) di dalam metode utama kelas Main, yang mencakup simulasi pendaftaran minimal tiga barang baru, operasi penambahan dan pengurangan stok yang berhasil, hingga pengujian validasi sistem yang dirancang untuk menolak transaksi ketika jumlah penarikan melebihi sisa stok yang tersedia di gudang.

2. Variabel

Nama Variabel	Tipe Data	Fungsi
idBarang	String	Menyimpan kode unik barang sebagai kunci pencarian utama.
namaBarang	String	Menyimpan nama deskriptif dari barang.
kategori	String	Menyimpan kelompok jenis barang untuk dikelompokkan di Set.
stok	int	Menyimpan jumlah kuantitas fisik barang di gudang.
databaseBarang	Map	Menyimpan seluruh objek barang berbasis ID untuk pencarian
kategoriUnik	Set	Menyimpan daftar kategori barang dan otomatis menolak nama ganda.
riwayat	List	Mencatat urutan kronologis string aktivitas transaksi gudang.

barangBaru	List	Menampung sementara objek barang yang baru dibuat sebelum disimpan.
barang	Barang	Memegang objek barang hasil pencarian dari Map untuk dimanipulasi.
stokLama	int	Menyimpan nilai stok sebelum ditambah atau dikurangi.
no	int	Membuat penomoranurut otomatis saat mencetak daftar riwayat.
kat, entry, b, log		Berfungsi sebagai variabel pembantu untuk membaca elemen koleksi saat cetak laporan.
args	String[]	Menampung argumen baris perintah bawaan Java saat program dijalankan.
gudang	SistemGudang	Menyimpan instans objek gudang untuk menggerakkan seluruh sistem simulasi.

3. Method dan Fungsi

Nama Method	Jenis Method	Fungsi
Barang()	Konstruktor	Mengisi nilai awal atribut ID, nama, kategori, dan stok saat objek dibuat.
getIdBarang()	Getter	Mengambil data ID barang untuk keperluan pencarian.
getNamaBarang()	Getter	Mengambil data nama barang untuk dicetak di laporan.
getKategori()	Getter	Mengambil data kategori barang.

getStok()	Getter	Mengambil jumlah sisa stok barang saat ini.
getStok(int Stok)	Setter	Mengubah atau memperbarui kuantitas stok barang.
SistemGudang()	Konstruktor	Menginisialisasi objek HashMap, HashSet, dan ArrayList di memori.
tambahBarangBaru()	Void	Memasukkan objek barang baru ke Map dan nama kategorinya ke Set.
tambahStok(id, jumlah)	Void	Menambah kuantitas stok barang di Map jika ID ditemukan, lalu mencatatnya ke riwayat.
kurangiStok(id, jumlah)	Void	Mengurangi stok barang di Map jika stok cukup, atau menolak transaksi jika stok kurang/ID tidak ada, lalu mencatatnya ke riwayat.
cetakLaporan()	Void	Menampilkan daftar kategori dari Set, sisa stok dari Map, dan urutan riwayat dari List ke layar.
main(String[] args)	Static	Menjalankan simulasi otomatis secara berurutan: mendaftarkan 3 barang, 1x tambah stok sukses, 1x kurang stok sukses, 1x kurang stok gagal, dan mencetak laporan akhir.

4. Penjelasan Program

Class Barang :

```
public class Barang {
```

Mendeklarasikan kelas publik bernama Barang.

```
private String idBarang;

    private String namaBarang;

    private String kategori;

    private int stok;
```

Deklarasi atribut objek secara private (enkapsulasi) untuk melindungi data ID, nama, kategori, dan kuantitas stok dari akses langsung luar kelas.

```
public Barang(String idBarang, String namaBarang, String
kategori, int stok) {

    this.idBarang = idBarang;

    this.namaBarang = namaBarang;

    this.kategori = kategori;

    this.stok = stok;

}
```

Konstruktor kelas untuk menginisialisasi nilai awal keempat atribut saat objek baru dibuat menggunakan kata kunci

```
public String getIdBarang() {

    return idBarang;

}

public String getNamaBarang() {

    return namaBarang;

}

public String getKategori() {

    return kategori;
```

```

    }

    public int getStok() {

        return stok;

    }

    public void setStok(int stok) {

        this.stok = stok;

    }

}

```

- Metode Getter (public) sebagai gerbang resmi untuk membaca nilai atribut yang terenkapsulasi dari luar kelas.
- Metode Setter (void) khusus untuk memperbarui nilai kuantitas stok barang secara dinamis.

Class SistemGudang :

```

import java.util.HashMap;
import java.util.HashSet;
import java.util.ArrayList;
import java.util.Map;
import java.util.Set;
import java.util.List;

```

Impor interface dan kelas JCF (Map, Set, List, HashMap, HashSet, ArrayList).

```

public class SistemGudang {

    private Map<String, Barang> databaseBarang;

    private Set<String> kategoriUnik;

    private List<String> riwayat;

```

Deklarasi tiga koleksi data non-statik sebagai atribut kelas:

- databaseBarang: Map pencarian cepat dengan ID sebagai kunci unik dan objek Barang sebagai nilai.
- kategoriUnik: Set penampung nama kategori untuk mencegah duplikasi data

- riwayat: List pencatat urutan kronologis string aktivitas transaksi.

```
public SistemGudang() {  
  
    this.databaseBarang = new HashMap<>();  
  
    this.kategoriUnik = new HashSet<>();  
  
    this.riwayat = new ArrayList<>();  
  
}
```

Konstruktor kelas SistemGudang untuk mengalokasikan memori awal bagi ketiga objek koleksi.

```
public void tambahBarangBaru(String id, String nama, String  
kategori, int stok) {  
  
    Barang barangBaru = new Barang(id, nama, kategori, stok);  
  
    databaseBarang.put(id, barangBaru);  
  
    kategoriUnik.add(kategori);  
  
    riwayat.add("Barang Masuk: " + id + " didaftarkan dengan  
stok " + stok + " unit.");  
  
}
```

Metode tambahBarangBaru : Membuat objek Barang baru, menyimpannya ke Map lewat .put(), memasukkan teks kategori ke Set lewat .add() , dan mencatat aktivitasnya ke List riwayat

```
public void tambahStok(String id, int jumlah) {  
  
    if (databaseBarang.containsKey(id)) {  
  
        Barang barang = databaseBarang.get(id);  
  
        int stokLama = barang.getStok();  
  
        barang.setStok(stokLama + jumlah);  
  
        riwayat.add("Barang Masuk: " + id + " ditambah " +  
jumlah + " unit.");  
  
    } else {  
  
        riwayat.add("Gagal!, ID " + id + " tidak  
ditemukan.");  
  
    }  
  
}
```

Metode tambahStok : Memvalidasi keberadaan ID via .containsKey(). Jika ada, mengambil objek via .get() , memperbarui total stok lewat gabungan getter-setter, lalu menambahkan catatan sukses atau gagal ke List riwayat.

```
public void kurangiStok(String id, int jumlah) {

    if (!databaseBarang.containsKey(id)) {

        riwayat.add("Gagal!, ID " + id + " tidak ditemukan.");

        System.out.println("Transaksi Gagal: ID Barang tidak ditemukan.");

        return;

    }

    Barang barang = databaseBarang.get(id);

    if (barang.getStok() >= jumlah) {

        int stokLama = barang.getStok();

        barang.setStok(stokLama - jumlah);

        riwayat.add("Barang : " + id + " dikurangi " + jumlah + " unit.");

    } else {

        riwayat.add("Gagal!, Sisa stok " + id + " tidak cukup untuk " + jumlah + " unit.");

        System.out.println("Transaksi Gagal: Stok tidak mencukupi.");

    }

}
```

Metode kurangiStok : Melakukan validasi awal ketersediaan ID; jika absen, catat kegagalan dan hentikan metode via return. Validasi kedua mengecek kecukupan kuantitas stok ; jika terpenuhi, stok dikurangi dan transaksi dicatat ke riwayat.

```
public void cetakLaporan() {

    System.out.println("\nLAPORAN AKHIR GUDANG");

    System.out.println("\n[DAFTAR KATEGORI BARANG]");

}
```



```

        for (String kat : kategoriUnik) {

            System.out.println("- " + kat);

        }

        System.out.println("\n[STATUS STOK BARANG SAAT INI]");

        for (Map.Entry<String, Barang> entry :
databaseBarang.entrySet()) {

            Barang b = entry.getValue();

            System.out.println("ID: " + b.getIdBarang() + " |
Nama: " + b.getNamaBarang() +
                                " | Kategori: " + b.getKategori()
+ " | Sisa Stok: " + b.getStok() + " unit");

        }

        System.out.println("\n[RIWAYAT AKTIVITAS TRANSAKSI]");

        int no = 1;

        for (String log : riwayat) {

            System.out.println(no + ". " + log);

            no++;

        }

    }
}

```

Metode cetakLaporan : Menggunakan perulangan for-each untuk mencetak daftar kategori unik dari Set, mencetak sisa stok setiap objek barang dari hasil penelusuran Map.entrySet(), serta menampilkan seluruh log aktivitas dari List dengan penomoran otomatis no++.

Class Main :

```

public class App {

    public static void main(String[] args) {

        SistemGudang gudang = new SistemGudang();
    }
}

```

Deklarasi kelas Main dan metode utama main sebagai titik awal eksekusi program Java, serta instansiasi objek gudang dari kelas SistemGudang yang otomatis menyiapkan penampung memori internal koleksi.

```
gudang.tambahBarangBaru("B01", "Laptop ASUS", "Elektronik", 10);  
gudang.tambahBarangBaru("B02", "Kursi Kerja", "Furnitur", 25);  
gudang.tambahBarangBaru("B03", "Mouse Logitech", "Elektronik",  
50);
```

Memanggil metode tambahBarangBaru secara hardcode untuk mendaftarkan 3 barang inventaris awal.

```
gudang.tambahStok("B01", 5);  
  
    gudang.kurangiStok("B02", 10);  
  
    gudang.kurangiStok("B01", 20);  
  
    gudang.cetakLaporan();  
  
    }  
  
}
```

- Simulasi 1x operasi penambahan stok yang berhasil pada item "B01"
- Simulasi 1x operasi pengurangan stok yang berhasil pada item "B02"
- Simulasi 1x operasi pengurangan stok yang dirancang gagal pada item "B01" karena jumlah penarikan (20) melebihi sisa stok saat itu.
- Memanggil metode cetakLaporan untuk menyajikan seluruh data rekapitulasi gudang di konsol, lalu mengakhiri jalannya program

5. Kesimpulan

Praktikum ini membuktikan bahwa integrasi ketiga pilar Java Collections Framework (JCF) memberikan solusi penyimpanan data dinamis yang sangat efisien untuk menggantikan array tradisional yang kaku. Penggunaan HashMap sebagai basis data utama terbukti efektif mempercepat proses pencarian dan pembaruan stok barang secara instan berdasarkan kunci ID unik dengan kompleksitas waktu konstan ($O(1)$), sementara implementasi HashSet berhasil menjamin keunikan rekapitulasi data karena karakteristiknya yang otomatis menolak nama kategori ganda. Selain itu, ArrayList sukses berfungsi sebagai buku besar yang merekam setiap urutan riwayat transaksi gudang secara terurut berdasarkan kronologi waktu kejadian, didukung oleh akurasi logika pengondisian sistem yang mampu memvalidasi dan menolak transaksi pengurangan stok secara tepat ketika jumlah penarikan melebihi sisa komoditas yang tersedia di gudang.

6. Daftar Pustaka

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). Modul Praktikum Pemrograman Berorientasi Objek: Java Collections. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.